Calidad visual y aprendizaje métrico con redes siamesas

Karen Sofía Delgado Calderon

Universidad de Cundinamarca

Especialización en Inteligencia Artificial

Deep Learning – Conceptos

R2-A1-S10 Redes Neuronales V

4 de noviembre de 2026

Calidad visual y aprendizaje métrico con redes siamesas

La calidad de una fotografía depende de su iluminación, contenido y resolución. Esta práctica describe tres imágenes públicas de figuras del ámbito musical, con fuentes y licencias verificables, y compara sus distribuciones de intensidad. El trabajo estudia la preparación de imágenes y desarrolla una revisión conceptual de las redes siamesas. El documento incorpora una adaptación práctica con dígitos para ejecutar Triplet Loss y evaluar one-shot y few-shot. El alcance se declara frente a la consigna original de la actividad 73148.

Selección y trazabilidad de las imágenes

Se seleccionaron tres archivos de Wikimedia Commons. La fotografía de Shakira corresponde a una rueda de prensa de 2014, mientras las de Juanes y Carlos Vives muestran escenas de concierto de 2015 y 2005, respectivamente (b r e n t, 2005; Casal Jr., 2014; Jörgens.Mi, 2015). Los nombres y contextos proceden de las fichas de origen. El manifiesto fuentes_publicas.json conserva autor, fecha, URL de descarga, página descriptiva y licencia.

Las dimensiones originales son 1875×2256, 5176×3556 y 602×419 píxeles. La muestra reúne escenas con contenido y condiciones diferentes. Se utiliza para describir variación fotográfica global. En particular, las escenas de concierto incorporan focos, instrumentos, fondo y varias personas. El cálculo de brillo de toda la imagen recoge esos elementos. La selección de tres archivos es una decisión de esta práctica y responde al alcance del análisis.

Preparación y medidas

Se corrigió la orientación EXIF, se convirtió cada archivo a RGB y se redujo su tamaño conservando la proporción, con un límite de 320×320 píxeles y remuestreo Lanczos. Los tamaños resultantes fueron 266×320, 320×220 y 320×223. ImageOps documenta las operaciones de orientación y reducción proporcional usadas en el script (Pillow contributors, s. f.).

Se calcularon las medidas sobre copias en escala de grises, con intensidades de 0 a 255.

Brillo corresponde a la media y contraste a la desviación estándar. Se eligieron límites de 10 y 245 para describir píxeles oscuros y claros, respectivamente. La entropía se obtuvo a partir del histograma de 256 valores. La varianza del Laplaciano discreto de cuatro vecinos se calculó sobre los píxeles interiores. Esta medida responde a bordes, textura, ruido y escala, por lo que requiere observar también la imagen.

Figura 1

Imágenes públicas y sus versiones en escala de grises



Nota. Imágenes 01, 02 y 03 atribuidas a Marcello Casal Jr./Agência Brasil, Jörgens.Mi/Wikipedia y b r e n t, con retoque de Resize. Reducción proporcional y conversión a grises realizadas para este análisis. Créditos, enlaces y licencias se detallan en la sección correspondiente. La adaptación de pública02 se distribuye bajo CC BY-SA 3.0.

Resultados

La Tabla 1 muestra la diferencia entre la toma cercana y las escenas de concierto. Pública01 tiene mayor brillo medio (135,93). Pública02 reúne un contraste mayor (65,44), un 13,53 % de

píxeles oscuros y un 1,87 % de píxeles claros bajo los límites elegidos. Los focos y las zonas oscuras de la escena contribuyen a ese contraste.

Tabla 1

Medidas globales después de la reducción proporcional

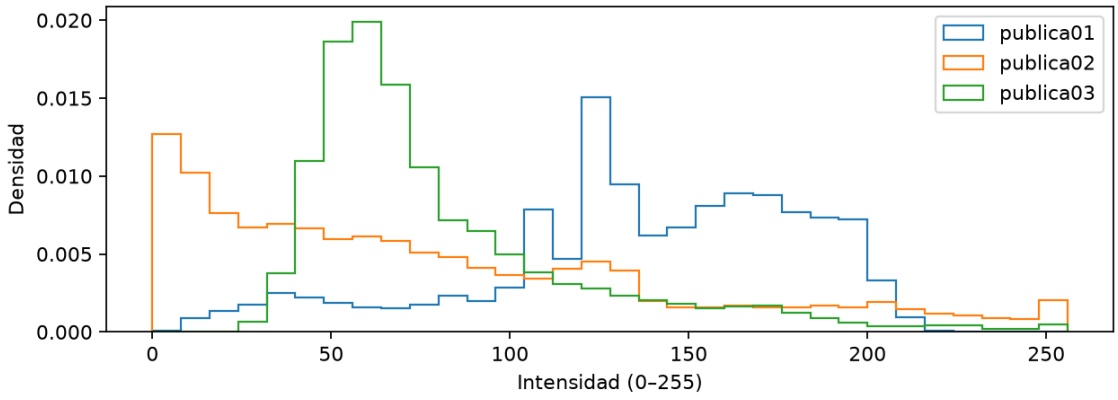
Archivo	Brillo	Contraste	Oscuros (%)	Claros (%)
publica01	135.93	45.71	0.16	0.00
publica02	78.94	65.44	13.53	1.87
publica03	81.41	40.61	0.00	0.46

Nota. Se analizaron todas las regiones de cada imagen reducida. Los píxeles oscuros cumplen $I \leq 10$ y los claros $I \geq 245$. El valor de claros de pública01 es 0,00117 % antes del redondeo. Datos de calidad_publicas.csv.

La entropía fue 7,29, 7,60 y 6,91 bits por píxel, y la varianza del Laplaciano fue 848,72, 2252,27 y 1760,05. Estas diferencias describen escenas distintas y requieren separar iluminación, detalle y contenido. El mayor Laplaciano de pública02 coincide con una escena con numerosos bordes y focos. Ese resultado por sí solo resulta insuficiente para clasificar su enfoque como superior.

Figura 2

Histogramas de intensidad de las imágenes públicas



Nota. Se usaron 32 intervalos con densidad normalizada. Cada distribución corresponde a la imagen completa reducida. Elaboración propia.

La Figura 2 permite relacionar las estadísticas con la distribución completa. La normaliza-

ción de la densidad admite tamaños de imagen diferentes. Una evaluación controlada de cambios de exposición o desenfoque requeriría repetir la misma escena manteniendo encuadre y escala. Como ejercicio posterior de calidad, podrían medirse versiones de una misma imagen con una transformación documentada.

Fundamentos de redes siamesas

Una arquitectura siamesa utiliza un codificador con parámetros compartidos sobre varias entradas. Las representaciones se sitúan en un mismo espacio y su relación se expresa mediante una función de distancia. FaceNet describe el aprendizaje de representaciones mediante tríos de ejemplos (Schroff et al., 2015).

Triplet Loss relaciona un ancla, un ejemplo positivo y uno negativo. Conceptualmente, $L = \max(0, d(a, p) - d(a, n) + m)$, donde m es el margen. La selección de tríos y la calidad de las etiquetas afectan al aprendizaje. Este fundamento se presenta como revisión de literatura.

One-shot describe la disponibilidad de una referencia por clase, y few-shot la de unos pocos ejemplos. Los prototipos resumen observaciones en un espacio aprendido (Snell et al., 2017). Zero-shot requiere información auxiliar para relacionar clases sin ejemplos de referencia. Estas condiciones definen protocolos de evaluación distintos. El análisis de las imágenes públicas describe su calidad global. La adaptación con dígitos se desarrolla a continuación.

Adaptación práctica con dígitos

La pregunta experimental es si una representación aprendida con Triplet Loss transfiere mejor a clases nuevas que una línea base de píxeles normalizados. Se utilizó el conjunto Digits, de imágenes de 8×8, incluido en scikit-learn (Scikit-learn developers, s. f.). Esta práctica implementa aprendizaje métrico con dígitos y se presenta como adaptación de la parte experimental de la consigna.

Las clases 0 a 4 aportaron 901 imágenes de ajuste. Se reservaron las clases 5 y 6 para calibración y las clases 7, 8 y 9 para prueba. Se normalizaron intensidades dividiendo por 16.

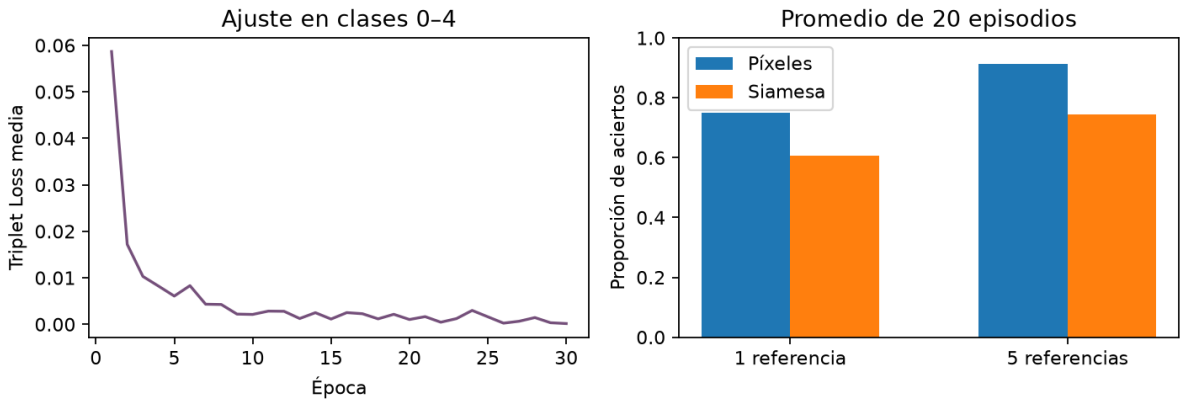
El codificador comparte pesos entre sus ramas y transforma las 64 entradas en una capa de 128 unidades con ReLU, seguida de 32 coordenadas normalizadas. Adam ajustó los parámetros con tasa 0,001, margen 0,25, 30 épocas y diez lotes de 64 tríos por época. Cada trío contiene dos muestras distintas de una clase de ajuste y una de otra clase. Se empleó TripletMarginLoss de PyTorch (PyTorch contributors, s. f.). La semilla fue 211.

One-shot utiliza una referencia y few-shot utiliza cinco. En cada condición se promedian y normalizan las referencias para obtener un prototipo. La línea base aplica el mismo procedimiento a los 64 píxeles normalizados. Los dos métodos evalúan exactamente las mismas galerías y consultas. Se reserva siempre un conjunto de cinco imágenes por clase para construir la galería, y otras 20 como consultas. Hay 20 episodios con distintas selecciones de referencias y consultas. Las imágenes pueden repetirse entre episodios, por lo que las 1.200 consultas acumuladas corresponden a evaluaciones reutilizadas, no a 1.200 imágenes únicas.

Los umbrales se eligieron en las clases de calibración minimizando FAR + FRR sobre 201 valores entre 0 y 2. Se congelaron antes de evaluar las clases de prueba. FAR cuenta aceptaciones entre clases distintas y FRR rechazos dentro de la misma clase. La asignación cerrada elige el prototipo más cercano y su exactitud se calcula por separado.

Figura 3

Ajuste de la red y comparación con una línea base



Nota. Pérdida media por época sobre las clases 0 a 4, y exactitud media de los 20 episodios sobre clases 7 a 9. Elaboración propia.

La pérdida desciende de 0,058657 a 0,000157. La Tabla 2 y la Figura 3 muestran que ese descenso no asegura una representación superior para clases reservadas. La línea base obtiene mayor exactitud en las dos condiciones de referencias. El entrenamiento se concentra en cinco clases, lo que puede limitar la transferencia a trazos de otros dígitos.

Tabla 2
Evaluación acumulada de los episodios de prueba

Representación	Referencias	Aciertos (%)	FAR (%)	FRR (%)
Píxeles	1	75,08	40,75	18,33
Píxeles	5	91,33	15,38	17,50
Siamesa	1	60,58	16,04	51,75
Siamesa	5	74,33	40,83	15,75

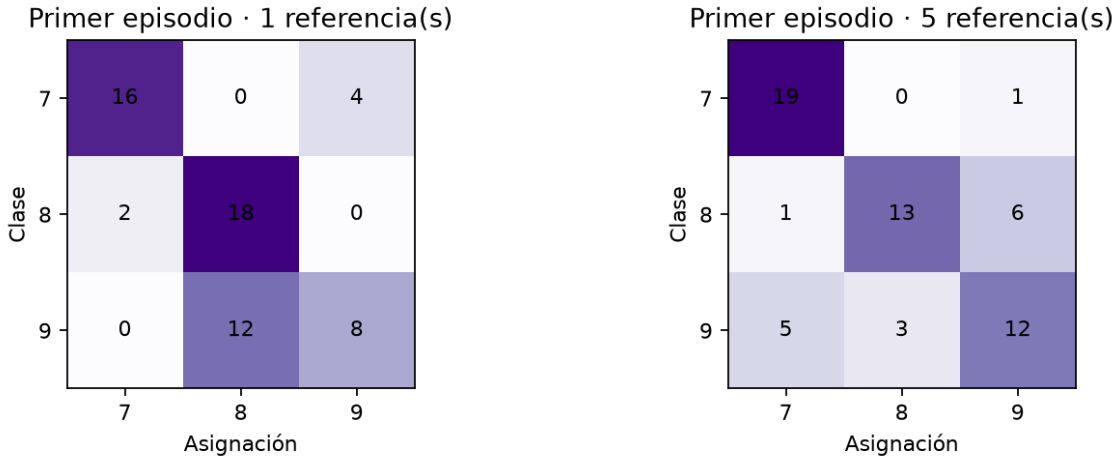
Nota. Cada fila reúne 1.200 consultas genuinas y 2.400 comparaciones entre clases distintas, con reutilización de imágenes entre episodios. Los umbrales de calibración fueron 0,75 y 0,59 para píxeles, y 0,88 y 1,01 para la red.

Fuente: resumen_metrica.json.

Cinco referencias aumentan la exactitud de la red, pero también su FAR. El prototipo y el umbral calibrado cambian entre condiciones. Esto ilustra que aumentar referencias puede desplazar los tipos de error y exige interpretar varias medidas. Los promedios describen una semilla y un conjunto pequeño de clases. Para estimar incertidumbre harían falta repeticiones del entrenamiento y particiones de clases independientes.

Figura 4

Asignaciones de la red en el primer episodio de prueba



Nota. Filas indican clase real y columnas la asignada. Se muestran 20 consultas por clase en el primer episodio. El resumen de la tabla utiliza los veinte episodios. Elaboración propia.

El cuaderno Karen_Siamesa_Digitos.ipynb incluye código, salidas y figuras. Se publica en el mismo repositorio del análisis de imágenes y puede abrirse en Google Colab. Zero-shot queda como explicación conceptual, ya que los episodios evaluados utilizan referencias visuales.

Créditos y condiciones de reutilización

publica01. Shakira. Marcello Casal Jr/Agência Brasil. 2014-07-12. Archivo *Shakira 2014.jpg*. CC BY 3.0 BR, https://creativecommons.org/licenses/by/3.0/br/. Fuente: https://commons.wikimedia.org/wiki/File:Shakira_2014.jpg. Transformaciones de esta práctica: reducción proporcional y conversión a grises.

publica02. Juanes. Jörgens.Mi/Wikipedia. 2015-07-18. Archivo *Juanes ZMF 2015 jm49211.jpg*. CC BY-SA 3.0, https://creativecommons.org/licenses/by-sa/3.0/. Fuente: https://commons.wikimedia.org/wiki/File:Juanes_ZMF_2015_jm49211.jpg. Transformaciones de esta práctica: reducción proporcional y conversión a grises.

publica03. Carlos Vives. b r e n t; retoque de Resize. 2005-09-16. Archivo *Carlos Vives.jpg*. CC BY 2.0, https://creativecommons.org/licenses/by/2.0/. Fuente: https://commons.wikimedia.org/wi

ki/File:Carlos_Vives.jpg. Transformaciones de esta práctica: reducción proporcional y conversión a grises.

Los archivos originales se incluyen en datos_publicos. El panel contiene reducciones y versiones en grises. La versión derivada de pública02 conserva CC BY-SA 3.0. Las fotografías 01 y 03 mantienen sus condiciones de atribución correspondientes. La selección y la publicación de resultados describen un uso académico de las imágenes.

Reproducción y alcance

El script calidad_publicas.py se ejecutó localmente el 4 de noviembre de 2026 y generó calidad_publicas.csv, calidad_publicas.json y las dos figuras. El cuaderno Karen_Calidad_Publicas.ipynb conserva las salidas numéricas. El código y los resultados se publican en https://github.com/sofiadelcal/karen-calidad-imagenes-publicas-s10. Para repetir el análisis se necesita Python con Pillow, NumPy y Matplotlib, el manifiesto y los tres archivos en datos_publicos. El programa descargar_publicas.py obtiene las copias incluidas en el repositorio. Las fuentes originales y sus licencias figuran en el manifiesto.

El trabajo desarrolla procedencia, licencias, preprocesamiento y calidad de imágenes públicas, y añade una red siamesa funcional con dígitos. Su implementación acredita entrenamiento, Triplet Loss y evaluación con pocas referencias. La consigna original exige reconocimiento facial, por lo que la aceptación de esta adaptación depende de la profesora. La práctica presentada mide la clasificación y comparación de dígitos.

Referencias

b r e n t. (2005, 16 de septiembre). *Carlos Vives.jpg* [Fotografía, versión retocada por Resize]. Wikimedia Commons. https://commons.wikimedia.org/wiki/File:Carlos_Vives.jpg

Casal Jr., M. (2014, 12 de julio). *Shakira 2014.jpg* [Fotografía]. Agência Brasil; Wikimedia Commons. https://commons.wikimedia.org/wiki/File:Shakira_2014.jpg

Jörgens.Mi. (2015, 18 de julio). *Juanes ZMF 2015 jm49211.jpg* [Fotografía]. Wikimedia Commons. https://commons.wikimedia.org/wiki/File:Juanes_ZMF_2015_jm49211.jpg

Pillow contributors. (s. f.). *ImageOps module*. Pillow documentation. https://pillow.readthedocs.io/en/stable/reference/ImageOps.html

PyTorch contributors. (s. f.). *TripletMarginLoss*. PyTorch documentation. https://docs.pytorch.org/docs/stable/generated/torch.nn.TripletMarginLoss.html

Schroff, F., Kalenichenko, D., & Philbin, J. (2015). FaceNet: A unified embedding for face recognition and clustering. *2015 IEEE Conference on Computer Vision and Pattern Recognition* (pp. 815–823). https://doi.org/10.1109/CVPR.2015.7298682

Scikit-learn developers. (s. f.). *load_digits*. scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html

Snell, J., Swersky, K., & Zemel, R. S. (2017). Prototypical networks for few-shot learning. *Advances in Neural Information Processing Systems, 30*. https://papers.nips.cc/paper/6996-prototypical-networks-for-few-shot-learning

Apéndice: código de la adaptación con dígitos

```
from pathlib import Path
import json
import numpy as np
import matplotlib.pyplot as plt
import torch
from torch import nn
from torch.nn import functional as F
from sklearn.datasets import load_digits
from sklearn.metrics import confusion_matrix

ROOT = Path(__file__).resolve().parent
OUT = ROOT / "resultados_metrica"
OUT.mkdir(exist_ok=True)
SEMILLA = 211
torch.manual_seed(SEMILLA)
torch.set_num_threads(2)
rng = np.random.default_rng(SEMILLA)
datos = load_digits()
pixeles = datos.data.astype(np.float32) / 16
etiquetas = datos.target
X = torch.tensor(pixeles)
grupos = {c: np.flatnonzero(etiquetas == c) for c in range(10)}
assert set(range(5)).isdisjoint([5,6,7,8,9])

# %% Codificador compartido
class Codificador(nn.Module):
    def __init__(self):
        super().__init__()
        self.capas = nn.Sequential(nn.Linear(64,128), nn.ReLU(), nn.Linear(128,32))
    def forward(self, lote):
        return F.normalize(self.capas(lote), dim=1)

red = Codificador()
opt = torch.optim.Adam(red.parameters(), lr=0.001)
perdida = nn.TripletMarginLoss(margin=0.25)
historial = []
for epoca in range(30):
    errores = []
    for paso in range(10):
        a, p, n = [], [], []
        for _ in range(64):
            ca, cn = rng.choice(5,2,replace=False)
            ia, ip = rng.choice(grupos[ca],2,replace=False)
            a.append(ia)
            p.append(ip)
            n.append(rng.choice(grupos[cn]))
        opt.zero_grad()
        costo = perdida(red(X[a]),red(X[p]),red(X[n]))
        costo.backward()
```

```
        opt.step()
        errores.append(costo.item())
    historial.append(float(np.mean(errores)))
red.eval()
with torch.no_grad():
    aprendidos = red(X).numpy()
crudos = pixeles / np.linalg.norm(pixeles,axis=1,keepdims=True)
assert np.isfinite(aprendidos).all()
assert np.allclose(np.linalg.norm(aprendidos,axis=1),1,atol=1e-5)
print("Imagenes de ajuste:", sum(len(grupos[c]) for c in range(5)))
print("Triplet Loss inicial/final:", historial[0], historial[-1])

# %% Protocolo de episodios
def episodio(clases, generador):
    soporte, consultas, verdad = [], [], []
    for c in clases:
        orden = generador.permutation(grupos[c])
        soporte.append(orden[:5])
        consultas.extend(orden[5:25])
        verdad.extend([c]*20)
    assert not set(np.ravel(soporte)).intersection(consultas)
    return np.asarray(soporte),np.asarray(consultas),np.asarray(verdad),np.asarray(clases
    )

def distancias(representaciones, episodio_actual, shots):
    soporte, consultas, verdad, clases = episodio_actual
    prototipos = representaciones[soporte[:,:shots]].mean(axis=1)
    prototipos /= np.linalg.norm(prototipos,axis=1,keepdims=True)
    d = np.linalg.norm(representaciones[consultas,None,:]-prototipos[None,:,:],axis=2)
    return d,verdad,clases

def medir(d, verdad, clases, umbral):
    iguales = verdad[:,None] == clases[None,:]
    pred = clases[d.argmin(axis=1)]
    return dict(aciertos=int((pred==verdad).sum()), consultas=len(verdad),
        FA=int((d[~iguales]<=umbral).sum()), diferentes=int((~iguales).sum()),
        FR=int((d[iguales]>umbral).sum()), iguales=int(iguales.sum()))

calibracion = episodio([5,6],np.random.default_rng(900))
episodios = [episodio([7,8,9],np.random.default_rng(1000+i)) for i in range(20)]
filas = []
for nombre, vectores in [("Pixeles",crudos),("Siamesa",aprendidos)]:
    for shots in [1,5]:
        d,v,c = distancias(vectores,calibracion,shots)
        umbrales = np.linspace(0,2,201)
        costos=[]
        for t in umbrales:
            m=medir(d,v,c,t)
            costos.append(m["FA"]/m["diferentes"]+m["FR"]/m["iguales"])
        umbral=float(umbrales[np.argmin(costos)])
        mediciones=[]
        for e in episodios:
            dt,vt,ct=distancias(vectores,e,shots)
            mediciones.append(medir(dt,vt,ct,umbral))
```

```
        aciertos=sum(m["aciertos"] for m in mediciones)
        total=sum(m["consultas"] for m in mediciones)
        fila=dict(metodo=nombre,referencias=shots,umbral=umbral,
            aciertos=aciertos,consultas=total,exactitud=aciertos/total,
            FAR=sum(m["FA"] for m in mediciones)/sum(m["diferentes"] for m in mediciones)
    ,
            FRR=sum(m["FR"] for m in mediciones)/sum(m["iguales"] for m in mediciones))
        filas.append(fila)
        if nombre=="Siamesa":
            dt,vt,ct=distancias(vectores,episodios[0],shots)
            cm=confusion_matrix(vt,ct[dt.argmin(axis=1)],labels=ct)
            fig,ax=plt.subplots(figsize=(3.8,3.2))
            ax.imshow(cm,cmap="Purples")
            for (i,j),valor in np.ndenumerate(cm):
                ax.text(j,i,str(valor),ha="center",va="center")
            ax.set(xticks=range(3),yticks=range(3),xticklabels=ct,yticklabels=ct,
                xlabel="Asignacion",ylabel="Clase",title=f"Primer episodio  {shots}
    referencia(s)")
            fig.tight_layout()
            fig.savefig(OUT/f"matriz_{shots}.png",dpi=160)
            plt.close(fig)
print(json.dumps(filas,indent=2,ensure_ascii=False))
(OUT/"resumen_metrica.json").write_text(json.dumps(dict(
    semilla=SEMILLA,clases_ajuste=[0,1,2,3,4],clases_calibracion=[5,6],
    clases_prueba=[7,8,9],episodios=20,historial=historial,resultados=filas),
    indent=2,ensure_ascii=False),encoding="utf8")
fig,axs=plt.subplots(1,2,figsize=(9,3.2))
axs[0].plot(range(1,31),historial,color="#75507b")
axs[0].set(xlabel="Epoca",ylabel="Triplet Loss media",title="Ajuste en clases 04")
pos=np.arange(2)
for i,nombre in enumerate(["Pixeles","Siamesa"]):
    valores=[f["exactitud"] for f in filas if f["metodo"]==nombre]
    axs[1].bar(pos+i*0.32,valores,width=0.32,label=nombre)
axs[1].set(xticks=pos+0.16,xticklabels=["1 referencia","5 referencias"],
    ylim=(0,1),ylabel="Proporcion de aciertos",title="Promedio de 20 episodios")
axs[1].legend()
fig.tight_layout()
fig.savefig(OUT/"comparacion_metrica.png",dpi=160)
plt.close(fig)
print("Experimento de digitos finalizado.")
```